Where NOBLIVION improves Claude Code memory

This page lists each area where NOBLIVION changes how Claude Code handles memory. For each area it says what Claude Code does without NOBLIVION, what NOBLIVION adds, and which setting controls it. The page describes NOBLIVION 0.1.23.

Claude Code's built-in memory is a folder of Markdown notes for each project and an index file, `MEMORY.md`. At the start of a session, Claude Code loads the first 200 lines or 25 KB of `MEMORY.md`. The model must then decide by itself to open a note. NOBLIVION keeps these files as they are. It never edits `MEMORY.md`. It adds a local store that indexes the notes, and hooks that bring the right note to the model at the moment it needs it.

Every setting below is described in full in configuration.md.

Summary

Area	Shipped state	Setting
1. Recall on every prompt	on	`NOBLIVION_RECALL_INDEX`
2. Label rows	on	`NOBLIVION_RECALL_LABELS`
3. Search and fetch tools	on	MCP tools `noblivion_recall`, `noblivion_remember`
4. Tool guard	on	guard table, note fields `triggers:` and `violates:`
5. Credential guard	on	always first, never quarantined
6. Error recall	on, `local` mode	`NOBLIVION_ERROR_RECALL_MODE`
7. Rules for subagents	on	`NOBLIVION_SUBAGENT_RULES_OFF`
8. Checks when a note is written	on	memory fields hook, `noblivion_remember`
9. Older notes and deleted notes	on	`recall.older_mark`, `index.delete_grace_days`
10. One store for many projects and Codex	on	`recall.shared_roots`, Codex adapter
11. Logs and diagnostics	on	`noblivion doctor`, `explain`, `stop report`, `trust report`
12. Session briefing and continuity	off	`NOBLIVION_CONTINUITY`
13. Trust in ranking	off	`trust.ranking`, `NOBLIVION_RECALL_INDEX_TRUST`
14. Stop checks	shadow (log only)	`stop.mode`, `stop.checks`, `stop.reminders`
15. Background learning	off until configured	`learning.json` in the store data folder
16. Guard rewrite and ask	off	`guard.rewrite`, `guard.ask`, `guard.act_shadow`
17. Recall reminder	off	`recall.nudge`
18. Deploy stop check	off until a host is named	`stop.deploy_hosts`
19. Warm prompt hook	off	file `warm-prompt-hooks` in the data folder
20. Duplicate sweep	manual, with consent	`dedup.model`, `noblivion dedup`

"Shipped state" is the state after a normal install with the shipped config file.

Areas that are on after install

1. Recall on every prompt

- **Without NOBLIVION.** Claude Code loads the start of `MEMORY.md` once, at session start. A note that the index does not name, or that is past the load limit, reaches the model only if the model opens it.
- **With NOBLIVION.** On every prompt, the prompt hook sends the prompt text to the store. The store ranks all notes against it. The hook shows the best rows before the model answers. A row carries the note's rule, and the best rows also carry its `apply:` text.
- **Why it helps.** The rule that fits the current task is in front of the model at the moment of the task, not only at session start.
- **Setting.** `NOBLIVION_RECALL_INDEX`. The shipped config file turns it on. `NOBLIVION_RECALL_INDEX_MAX_CHARS` limits the size of the rows. See how-recall-works.md.

2. Label rows

- **Without NOBLIVION.** A note about a file, a tool or a ticket is found only through the index or by a search the model starts.
- **With NOBLIVION.** When the prompt names a file, a tool, a ticket or a service, the hook adds rows for the notes that name the same thing.
- **Why it helps.** A note about a named object reaches the model even when the words of the prompt do not match the words of the note.
- **Setting.** `NOBLIVION_RECALL_LABELS`. The shipped config file turns it on.

3. Search and fetch tools

- **Without NOBLIVION.** The model reads notes with file tools. It must know the file name, or search the folder with text tools.
- **With NOBLIVION.** The MCP tool `noblivion_recall` searches every note in every memory folder of the store. This includes notes that no index file links. A call with `fetch_id` returns the full note, or one section of it. The MCP tool `noblivion_remember` saves a note with the write checks of area 8.
- **Why it helps.** A note that is not in the index stays findable.
- **Setting.** On when the plugin is installed. See recall-reflex.md.

4. Tool guard

- **Without NOBLIVION.** A rule in a note is guidance. Nothing checks a command against it.
- **With NOBLIVION.** Before a tool call, the guard hook compares the call with the guard table. The table comes from the `triggers:` and `violates:` fields of the notes.
  - **Rows.** When a note's trigger fits the command, the hook shows the note's rule next to the call.
  - **Deny.** When a note's `violates:` pattern matches the command, the hook blocks the call and gives the rule as the reason.
  - **Quarantine.** A deny rule that users override often shows as a row only, and the session-start line names it.
- **Why it helps.** A rule acts at the command that would break it.
- **Setting.** On. See guards.md for the fields and for how to turn a guard off.

5. Credential guard

- **Without NOBLIVION.** Nothing stops a command that prints a secret into the conversation.
- **With NOBLIVION.** The credential guard runs before the rule checks. It blocks a command that would print a secret, or print a file from a closed list of secret sources.
- **Why it helps.** A secret does not enter the transcript or the model context.
- **Setting.** On. It is never quarantined. See "The credential guard" in guards.md.

6. Error recall

- **Without NOBLIVION.** When a command fails, the model sees the error. A note that describes the same error is found only if the model searches for it.
- **With NOBLIVION.** After a failed Bash call, the hook shows notes that quote words from the error text.
- **Why it helps.** A known fix for a known error reaches the model at the failure.
- **Setting.** `NOBLIVION_ERROR_RECALL_MODE`. `local` (the default) searches the memory files with a keyword score and needs no store. `store` asks the store. `fused` uses both. Error recall shows memory files only.

7. Rules for subagents

- **Without NOBLIVION.** A subagent does not get the main session's memory. Only a fork does.
- **With NOBLIVION.** When Claude Code starts a subagent, the hook ranks the notes against the task and adds the best rows to the subagent's context.
- **Why it helps.** A subagent follows the same rules as the main session.
- **Setting.** On. `NOBLIVION_SUBAGENT_RULES_OFF` turns it off. `NOBLIVION_SUBAGENT_RULES_ROWS` sets the number of rows (default 3). `NOBLIVION_SUBAGENT_RULES_MODE` is `context` (default) or `rewrite`.

8. Checks when a note is written

- **Without NOBLIVION.** Claude Code writes the note file as given. Nothing checks its fields or its content.
- **With NOBLIVION.**
  - The memory fields hook checks a note after it is written. A feedback note must carry `rule:`, `apply:`, `scope:` and `triggers:`. The hook reports a missing or bad field, so the model fixes the file.
  - `noblivion_remember` hides secrets in the text before it saves.
  - It accepts `supersedes:`, so a new note replaces old notes in one step.
  - It names similar notes that already exist.
  - It indexes the new note at once.
- **Why it helps.** Notes keep the fields that the guard and recall need, and the folder gets fewer duplicates and no secrets.
- **Setting.** On. See "The memory fields hook" in guards.md.

9. Older notes and deleted notes

- **Without NOBLIVION.** Two notes can give different rules for the same command, and nothing shows which is newer. A deleted note is gone at once.

- **With NOBLIVION.**
  - When two shown rows are notes that share two or more trigger items, the hook marks the older one.
  - A deleted note stays in the database for a grace period before the store removes it.
  - The rank of a note does not fall with its age alone.
- **Why it helps.** The model sees which of two related rules is the current one, and a delete by mistake can be undone.
- **Setting.** `recall.older_mark` (on). `index.delete_grace_days` (default 14).

10. One store for many projects and Codex

- **Without NOBLIVION.** Each project folder has its own memory. A note from one project is not found in another.
- **With NOBLIVION.**
  - One store indexes every memory folder.
  - Notes under a shared root can be found from every project. The default shared root is the home-directory project.
  - The Codex adapter gives Codex the same store, the same memory files, the two MCP tools, and hooks for prompt recall, guards and continuity.
- **Why it helps.** A rule learned in one project or one tool applies in all of them.
- **Setting.** `recall.shared_roots` (`[]` turns the default off). See shared-memory.md and codex.md.

11. Logs and diagnostics

- **Without NOBLIVION.** No record shows why a note was or was not used.
- **With NOBLIVION.** Each hook writes a decision log. The CLI reads them:
  - `noblivion doctor` checks the install, the settings and the store.
  - `noblivion explain` shows why a note did or did not show for a prompt.
  - `noblivion stop report` counts what each stop check did or would do.
  - `noblivion trust report` lists notes to retire, promote or demote.
- **Why it helps.** You can find and fix a rule that does not fire, or one that fires too often.
- **Setting.** On. See diagnostics.md.

Areas that you switch on

12. Session briefing and continuity

- **Without NOBLIVION.** At session start the model gets only the start of `MEMORY.md`. When Claude Code compacts the context, the rules shown before can be lost.
- **With NOBLIVION.**
  - At session start, a briefing shows open decisions, open items and traps. It takes them from the files in `continuity.decision_files` and `continuity.open_files`, and skips notes that `MEMORY.md` already links.
  - Before a compaction, the hook saves the rules that were shown. After the compaction, it prints them again.
- **Why it helps.** Work that runs past a compaction keeps its rules.
- **Setting.** `NOBLIVION_CONTINUITY` (off by default).

13. Trust in ranking

- **Without NOBLIVION.** All notes count the same.
- **With NOBLIVION.** The store records when a note is shown, when it is used and when a correction contradicts it. From these records it computes a trust value for each note. With trust in ranking on, the value changes the order of recall.
- **Why it helps.** Notes that are used rise, and notes that are contradicted fall.
- **Setting.** `trust.ranking` (`off`, `shadow` or `on`; default `off`), together with `NOBLIVION_RECALL_INDEX_TRUST`. Trust data stays on your machine. See trust.md.

14. Stop checks

- **Without NOBLIVION.** Claude Code can end a turn with work left open.
- **With NOBLIVION.** When Claude Code wants to end a turn, the stop hook runs the checks in the table below.

Check	Fires when
`commit`	A worktree that Claude Code edited in this turn has changes that are not committed.
`tests`	Code files changed and no test command ran after the last change.
`notify`	The reply says work was deferred, skipped or blocked, and no push notification was sent.
`deploy`	A pull request was merged and no check of a deploy host ran.
`lesson`	The user corrected Claude Code and no memory note with `rule:` and `apply:` was written.

In `enforce` mode, `commit`, `tests` and `deploy` send Claude Code back to work with a reason. `notify` and `lesson` only remind: the next prompt gets the reminder once.

- **Why it helps.** A turn does not end with uncommitted work, untested code or a lost lesson.
- **Setting.** `stop.mode`: `shadow` (default) runs the checks and logs what would have blocked, but never blocks. `enforce` lets them act. `stop.checks` picks the checks. `stop.reminders` picks the checks that only remind. See "Stop checks" in guards.md.

15. Background learning

- **Without NOBLIVION.** A note exists only when the model or the user writes it.
- **With NOBLIVION.** The learner turns new, eligible conversations into memories, inside a scope that you define. It uses a local model. Each step (capture, extraction, validation, storage, indexing) keeps its own state, so a failed step can run again.
- **Why it helps.** Lessons from past work become notes without a manual step.
- **Setting.** Off until you create `learning.json` in the store data folder, with a scope and a running local model. Each client needs explicit permission to contribute. `"enabled": false` in the store policy turns it off. See learning.md.

16. Guard rewrite and ask

- **Without NOBLIVION.** No rule changes a command.
- **With NOBLIVION.**
  - **Rewrite.** When a note's `violates:` pattern matches and the note has a valid `rewrite:` field, the guard runs the command in the shape of the rule instead of

blocking it. By default the user approves the new command first (`guard.rewrite_decision ask`).
    - **Ask.** A note marked `on_violation: ask` makes the guard ask the user, with the rule as the reason, instead of blocking.
    - **Shadow.** With `guard.act_shadow`, the guard only logs what it would rewrite or ask, and blocks as before.
- **Why it helps.** A rule can correct a command instead of only stopping it.
- **Setting.** `guard.rewrite`, `guard.ask` and `guard.act_shadow` (all off). See "Rewrite and ask" in guards.md.

17. Recall reminder

- **Without NOBLIVION.** Nothing reminds the model to check memory before an action that is hard to undo.
- **With NOBLIVION.** Before a matching call, the hook adds one line: call `noblivion_recall` first. The default matches are ticket create and comment tools, `gh pr merge` and `glab mr merge`. The line shows only when the session made no recall call in its last 10 tool calls.
- **Why it helps.** Memory is checked before a merge or a ticket, where a forgotten rule costs the most.
- **Setting.** `recall.nudge` (off). `recall.nudge_shadow` logs only. `recall.nudge_tools` and `recall.nudge_commands` set the matches. See recall-reflex.md.

18. Deploy stop check

- **Without NOBLIVION.** A merge can end a turn with no check that the change is live.
- **With NOBLIVION.** The `deploy` stop check (area 14) fires when a pull request was merged and no command checked a deploy host.
- **Setting.** Off while `stop.deploy_hosts` is empty. List your deploy hosts to turn it on.

19. Warm prompt hook

- **Without NOBLIVION.** Not applicable.
- **With NOBLIVION.** By default the prompt hook starts one process for each prompt. The warm path answers the prompt from the running MCP server instead, so no new process starts.
- **Setting.** Off. Create the file `warm-prompt-hooks` in the plugin data folder to turn it on. Remove the file to turn it off.

20. Duplicate sweep

- **Without NOBLIVION.** Duplicate notes stay until someone finds them.
- **With NOBLIVION.** `noblivion dedup plan` finds candidate pairs and asks a judge model which note to keep. The apply step moves the other note to an archive folder. A restore puts it back byte for byte.
- **Why it helps.** The folder stays smaller and recall shows one rule instead of two.
- **Setting.** Manual only. The judge is a hosted model, so the two notes of each pair leave your machine. You must set `dedup.model` and an API key, and type your consent. Without a model, `noblivion dedup plan` refuses. See dedup-and-privacy.md.

Not part of NOBLIVION

Transcript miner

NOBLIVION does not mine conversation transcripts. There is no `mine` command. Error recall shows memory files only. The setting `NOBLIVION_MINER` has no effect. Background learning (area 15) is the way NOBLIVION turns conversations into notes.